

System Integration Practices



Data Integration

Instructor: Huy Nguyen Dang Quang, Msc

Email: huyndq@duytan.edu.vn

REF: Shawn A. Butler, Ph.D

Senior Lecturer, Executive Education Program

Institute for Software Research

Carnegie Mellon University

Objectives

- Understand why data integration is so challenging
- Techniques for data integration
- Understand performance tradeoffs in data integration

Two Architectures For Data Integration

"Two Architectures For Data Integration" are two different approaches or methodologies for building applications that integration information from multiple sources.

Two common architectures for data integration:

ETL (Extract, Transform, Load):

- **Extract:** Data is extracted from various sources such as databases, text files, CRM systems, ERP systems, or web data,...
- **Transform:** Transformation steps may include *filtering, processing, format conversion, validity checking, and data normalization.*
- **Load:** The loading process may involve updating new data, updating existing data, or appending new data to the data warehouse.

ETL is often oriented towards big data, involving the extraction and processing of data within predefined time intervals (e.g., nightly, hourly). This approach is suitable for scenarios where data consistency and reliability are critical, such as business intelligence applications and reporting.

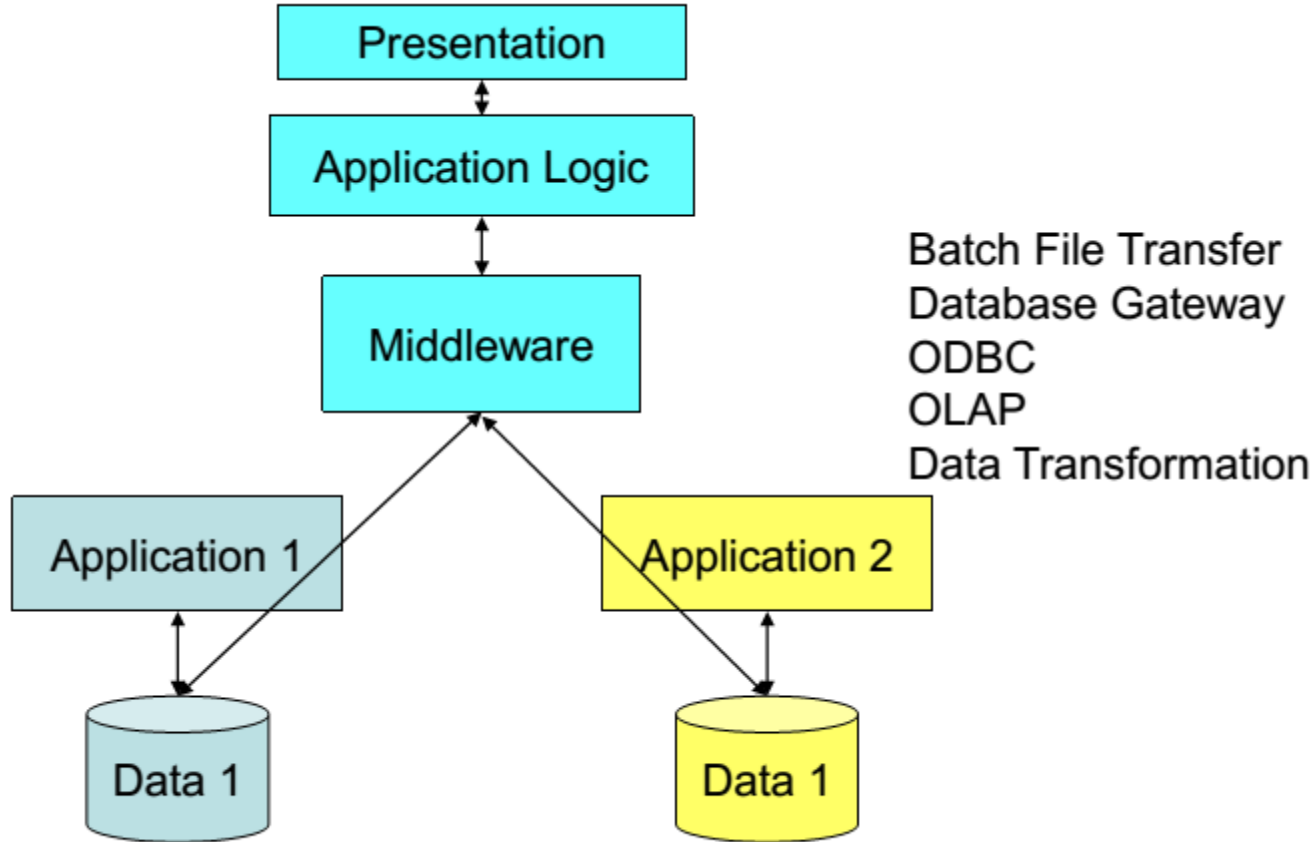
Two Architectures For Data Integration

ELT (Extract, Load, Transform):

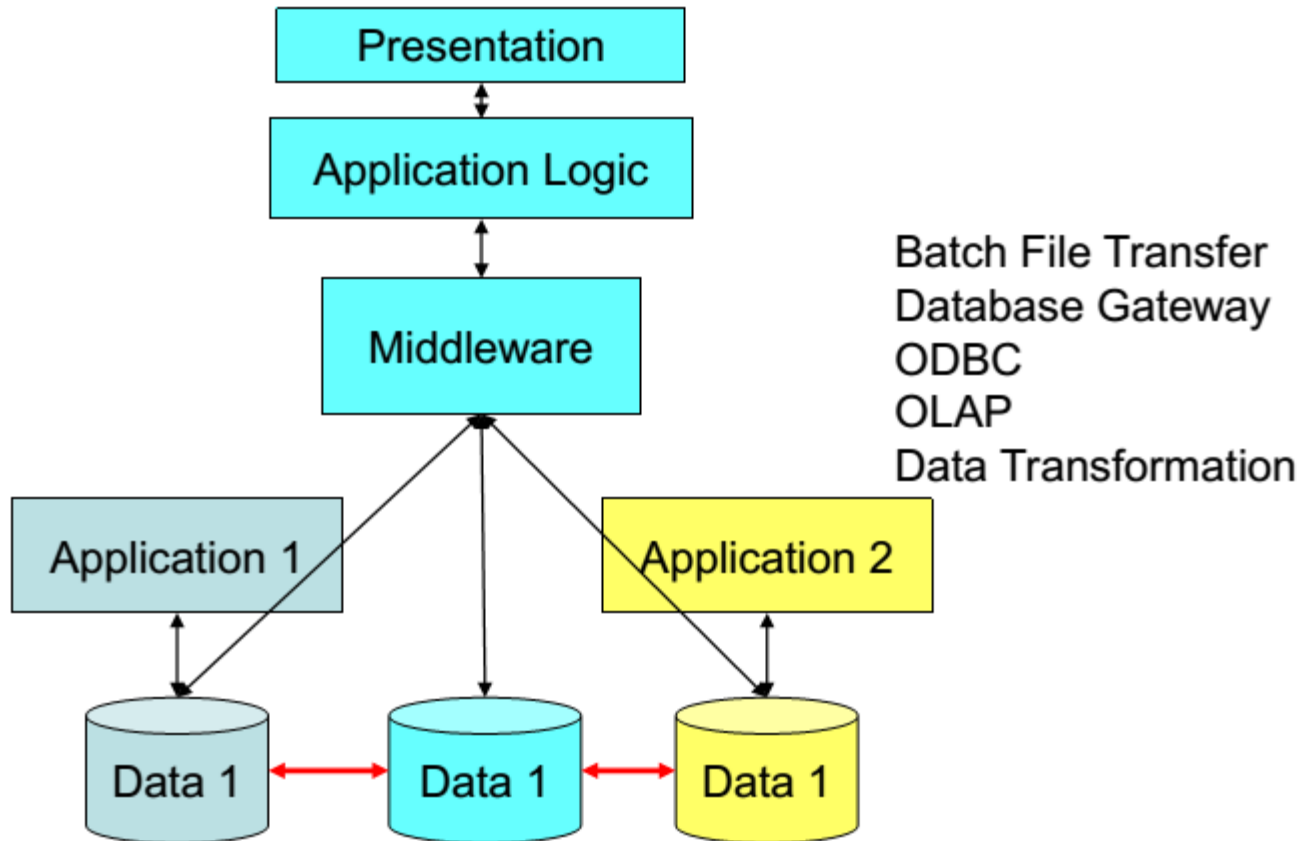
- ❑ **Extract:** Data is first extracted from the source systems without undergoing any significant transformation.
- ❑ **Load:** The extracted data is then loaded directly into the target data storage, such as a data lake or data warehouse, in its raw form.
- ❑ **Transform :** Once the data is loaded into the target storage, transformation and processing are performed within the target environment. This can involve various data manipulation techniques, such as data cleansing, enrichment, and aggregation.

ELT is used in big data environments where the target storage system has powerful processing capabilities. This approach offers greater flexibility and scalability, as transformations can leverage distributed computing resources within the storage environment.

Data Integration Model I



Data Integration Model II



2 Architecture-Many Choices

Comparison with Data Integration Model II

Aspect	Model I	Model II
Data Synchronization	None, separate	Two-way synchronization (red arrows)
Middleware Processing	Connection only	Connection + coordination between data copies
Data Integrity	Prone to conflicts	Higher if synchronization is effective
Scalability	High (but lacks data linkage)	More complex, but better data efficiency

2 Architecture-Many Choices

- Get all the records and aggregate manually
- Aggregate field in a special top level record, application making changes update the field
- System updates the aggregate field in system business logic
- The new application computes the aggregate on retrieval
- Special application “query processor” aggregates the information when needed
- Interface provides this as part of the system

Data Integration Common Mistakes

- Creating yet another database
- Waiting for the data analyst to finish developing the perfect schema
- Assuming the data exists as described in the documentation
- Implementing the perfectly normalized schema
- Testing without a sufficient set of real data
- Assuming that one site is a good representation of data at all sites

What is One Thing?

- We exist in a world of ambiguity
- The system cannot tolerate ambiguity
 - Oneness
 - Sameness – When are two things the same?
 - What is it? – In what categories does it exist?

In summary, in a world where ambiguity and diversity frequently exist, systems must be capable of ensuring clarity and consistency in data and information to operate efficiently. This raises the question of defining consistency and classifying information into clear categories.

Meaning

■ *What is a warehouse?*

- A location within a building?
- One physical building?
- Several physical buildings at a single location?
- A logical concept of where things are stored?

All definitions may be correct, but different among applications.

Meaning

■ *What is a street?*

- Segments along the physical road may have different names
- Different streets may have the same name
- Some roads have discontinuous segments
- Is a street terminated by a city, county, state?
- Does street imply motor traffic?
- Does it also mean freeways, highways, expressways, toll ways, circles, etc.

Categories, Attributes, Relationships

■ Categories (aka types)

- Require arbitrary decisions
- Categories often have subsets
- Overlap with other categories
 - ✓ Plaintiffs are people, corporations, government agencies...)
- Once categories are established real things are assigned
- Employee (part time, fulltime, managers, etc...)
- Items change in a category, does it still fit the category

Categories, Attributes, Relationships

- **Attributes** may be part of the category, or they may be an essential part of the categorization
 - Cars, by the way, have wheels
 - Not all cars have wheels
- **Relationships**
 - Relationship is not meaningful between things within a category
 - Relationship is meaningful, but cannot be related to itself
 - Relationship is meaningful, and things can be related to themselves

In summary, it emphasizes the diversity and complexity of attributes and relationships, as well as the dependency on context and how we define and use them in the classification and description of data.

Relational Databases

- Database normalization rules are designed to prevent anomalies and inconsistencies in databases
- Database normalization rules, strictly applied, may introduce inefficiencies in database design
- All databases have various schemas that work well for their application, but don't combine well into an efficient schema for all applications
- Database design is always a tradeoff among application performance optimization

Database Normalization

- The main goal of Database Normalization is to restructure the logical data model of a database to:
 - Eliminate redundancy
 - Organize data efficiently
 - Reduce the potential for data anomalies.

Example

A university stores student and class information in a single table:

STUDENT_ID	FULLNAME	CLASS_NAME	INSTRUCTOR_NAME
SV001	An	KTPM01	Thầy Nam
SV002	Bình	KTPM01	Thầy Nam
SV003	Cường	QTKD01	Cô Hoa

What problems may arise?

Example

- For example, with the database table of invoices (INVOICE) for customers below, what anomalies can be identified?

Acme Industries INVOICE				
Customer Number: 1454837			Terms: Net 30	
Customer: W. Coyote			Ship Via: USPS	
General Delivery				
Falling Rocks, AZ 84211			Order Date: 12/01/2008	
(599) 555-9345				
<u>Product No.</u>	<u>Description</u>	<u>Quantity</u>	<u>Unit Price</u>	<u>Extended Amount</u>
SPR-2290	Super Strength Springs	2	24.00	\$48.00
STR-67	Foot Straps, leather	2	2.50	\$5.00
HLM-45	Deluxe Crash Helmet	1	67.88	\$67.88
SFR-1	Rocket, solid fuel	1	128,200.40	\$128,200.40
ELT-1	Emergency Location Transmitter	1	79.88	**FREE GIFT**
TOTAL ORDER AMOUNT:				\$128,321.28

Example

- **Insertion anomaly:** A customer cannot be added to the database if they have not purchased any items.
- **Update anomaly:** To change a customer's address, it is necessary to update the address on all invoices for that customer because the customer's address is redundantly stored in all of their invoices.
- **Deletion anomaly:** If the last invoice of a customer is deleted, all data about that customer is lost.

The goals of normalization:

The objective of data normalization is to avoid anomalies.

- **Insertion anomaly:** Adding new rows may result in redundant data.
- **Deletion anomaly:** Deleting rows may cause the loss of necessary data for other rows.
- **Update anomaly:** Changing data in one row requires updating data in other rows due to redundancy.

The goals of normalization:

- The invoice table (INVOICE) above is represented in tabular form as follows.
- This table is in an unnormalized form.

INVOICE

Customer Number	Customer	Terms	Ship Via	Order Date	Product No.	Description	Quantity	Unit Price	Extended Amount
1454837	W.Coyote	Net 30	USPS	12/01/2008	SPR-2290	Super Strength Springs	2	24.00	\$48.00
1454837	W.Coyote	Net 30	USPS	12/01/2008	SRT-67	Foot Straps, leather	2	2.50	\$5.00
1454837	W.Coyote	Net 30	USPS	12/01/2008	HLM-45	Deluxe Crash Helmet	1	67.88	\$67.88
1454837	W.Coyote	Net 30	USPS	12/01/2008	SFR-1	Rocket, Solid fuel	1	128,200.4	\$128,200.4
1454837	W.Coyote	Net 30	USPS	12/01/2008	ELT-1	Emergency Location Transmitter	1	79.88	\$79.88

The goals of normalization:

- This table is normalized by splitting it into two tables as follows:

Customer Number	Customer	Terms	Ship Via	Order Date
1454837	W.Coyote	Net 30	USPS	12/01/2008

Customer Number	Product No.	Description	Quantity	Unit Price	Extended Amount
1454837	SPR-2290	Super Strength Springs	2	24.00	\$48.00
1454837	SRT-67	Foot Straps, leather	2	2.50	\$5.00
1454837	HLM-45	Deluxe Crash Helmet	1	67.88	\$67.88
1454837	SFR-1	Rocket, Solid fuel	1	128,200.4	\$128,200.4
1454837	ELT-1	Emergency Location Transmitter	1	79.88	\$79.88

Some forms of normalization

- **Some forms of normalization**

Common forms of data normalization include:

- **First Normal Form (1NF):** Eliminates repeating groups.
- **Second Normal Form (2NF):** Removes partially dependent fields from the primary key.
- **Third Normal Form (3NF):** Eliminates fields that are only dependent on another field.

Example 1: Non-Normalized Form

- Consider the following data table:

ĐƠN VỊ	Mã số ĐV	Tên ĐV	Mã số NQL	Địa điểm
	5	Nghiên cứu	NV002	Nam định, Hà nội, Bắc ninh
	4	Hành chính	NV014	Hà nội
	1	Lãnh đạo	NV061	Hà nội

- We see that the **Location field** of unit *5 contains multivalued attributes (more than one value)* with three places: *Nam Định, Hà Nội, Bắc Ninh*. Therefore, this **data table is not in a normalized form.**

Example 2: Non-Normalized Form

- Consider the following data table:

NHÂNVIÊN_DỰÁN	Mã số DA	Tên DA	Tên nhân viên	Số giờ
	1	DA01	Vân	15
			Nam	20
	2	DA02	Nam	10
			Thanh	12
			Bằng	28
	3	DA03	Thanh	20
			Vân	12

- We see that the attribute group {EmployeeName, Hours} is repeated multiple times across different projects. Therefore, this database is not in a normalized form.

Example 3: Non-Normalized Form

- Consider the following data table:

Customer Number	Customer	Terms	Ship Via	Order Date	Product No.	Description	Quantity	Unit Price	Extended Amount
1454837	W.Coyote	Net 30	USPS	12/01/2008	SPR-2290	Super Strength Springs	2	24.00	\$48.00
1454837	W.Coyote	Net 30	USPS	12/01/2008	SRT-67	Foot Straps, leather	2	2.50	\$5.00
1454837	W.Coyote	Net 30	USPS	12/01/2008	HLM-45	Deluxe Crash Helmet	1	67.88	\$67.88
1454837	W.Coyote	Net 30	USPS	12/01/2008	SFR-1	Rocket, Solid fuel	1	128,200.4	\$128,200.4
1454837	W.Coyote	Net 30	USPS	12/01/2008	ELT-1	Emergency Location Transmitter	1	79.88	\$79.88

- This is an invoice for a customer. *If there is another invoice, the attributes {Product No, Description, Quantity, Unit Price, Extended Amount} will repeat in other invoices.* Therefore, this table is not in a normalized form.

First Normal Form (1 NF)

Concept: A relational schema (R) is called in First Normal Form (1NF) if all attributes of R are atomic attributes. A database schema is said to be in First Normal Form if all of its sub-relational schemas R_i are in First Normal Form.

First Normal Form (1 NF)

- Example 1: A relational schema in First Normal Form (1NF).

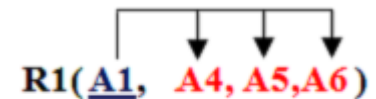
<u>MSKH</u>	TÊNKH	TP	PVC	<u>MSMH</u>	TÊNMH	ĐG	SL
S1	An	HCM	01	P1	Táo	650	300
S1	An	HCM	01	P2	Chanh	500	200
S1	An	HCM	01	P3	Táo	450	400
S2	Hòa	HN	02	P1	Chanh	650	100
S2	Hoà	HN	02	P3	Táo	450	300
S3	Thanh	NT	03	P2	Chanh	500	200
S4	Trang	NT	03	P2	Chanh	500	210

First Normal Form (1 NF)

- **General Principle:** Eliminate repeating or multivalued attributes.
- **Implementation Steps:**
 1. Separate the group of repeating/multivalued attributes into a new table.
 2. The key of the new table is the key of the original table and the key of the repeating group.
 3. The remaining table contains the key and the other attributes.



- **Example:** Given the relation
- With $A1$ is the primary key of the relation, and $\{A4, A5, A6\}$ are the repeating attributes.
The relation R will be split into two relations, $R1$ and $R2$.



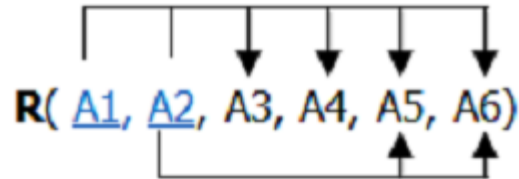
Second Normal Form (2 NF)

- The relation R is said to be in Second Normal Form (2NF) if it is already in First Normal Form (1NF) and all non-key attributes are fully functionally dependent on the key.
- Non-key attributes are those that do not participate in any key.

Second Normal Form (2 NF)

- **Step 1:** Remove the non-key attributes that depend on a part of the primary key and separate them into a new table, where the primary key of the new table is the part of the key they depend on.
- **Step 2:** The remaining attributes form a relation, with its primary key being the original primary key.

Second Normal Form (2 NF)



- The **primary key** is the set $\{A1, A2\}$ (i.e., $\{A1, A2\} \rightarrow \{A3, A4, A5, A6\}$).
- There is one functional dependency that partially depends on the key: $\{A2\} \rightarrow \{A5, A6\}$.
- The relation is split into the following two relations:
 - $R1(A2, A5, A6)$ Separates the attributes that depend on a part of the key
 - $R2(A1, A2, A3, A4)$ The remaining original table

Second Normal Form (2 NF)

- Ví dụ: cho quan hệ **R**(MSKH, TÊNKH, TP, PVC, MSMH, TÊNMH, ĐG, SL) và tập phụ thuộc hàm F được định nghĩa trên R:

- Tập phụ thuộc hàm:

F = {
 MSKH → TÊNKH, TP
 MSMH → TÊNMH, ĐG
 MSKH, MSMH → SL
 TP → PVC}

<u>MSKH</u>	TÊNKH	TP	PVC	<u>MSMH</u>	TÊNMH	ĐG	SL
S1	An	HCM	01	P1	Táo	650	300
S1	An	HCM	01	P2	Chanh	500	200
S1	An	HCM	01	P3	Cam	450	400
S2	Hòa	HN	02	P1	Táo	650	100
S2	Hoà	HN	02	P3	Cam	450	300
S3	Thanh	NT	03	P2	Chanh	500	200
S4	Trang	NT	03	P2	Chanh	500	210

- Quan hệ này được tách thành các quan hệ sau
- R1(MSKH, TÊNKH, TP, PVC)
- R2(MSKH, MSMH, TÊNMH, ĐG, SL)
- Trong R2 chứa phụ thuộc hàm {MSMH} → {TÊNMH, ĐG} tách thành
- R21(MSMH, TÊNMH, ĐG)
- R22(MSKH, MSMH, SL)

Third Normal Form (3 NF)

- Given the relation R and F as the set of functional dependencies defined on R.
- R is said to be in Third Normal Form (3NF) if it satisfies the following conditions:
 - R must be in Second Normal Form (2NF).
 - All non-key attributes must not be transitively dependent on the key (i.e., all attributes must be directly inferred from the key).

Third Normal Form (3 NF)

Example: Relationship R (MSSV, TenSV, MaLop, TenLop, MaMH, TenMH, Diem)

Primary key: (MSSV, MaMH)

$F_{R1} = \{\mathbf{MSSV}\} \rightarrow \{\text{TenSV, MaLop}\}$

$F_{R2} = \{\text{MaLop} \rightarrow \text{TenLop}\}$

$F_{R3} = \{\text{MaMH} \rightarrow \text{TenMH}\}$

The relation **is not in 3NF (Third Normal Form)** because:

There is a transitive dependency:

$\text{MSSV} \rightarrow \text{MaLop}$ and $\text{MaLop} \rightarrow \text{TenLop}$

$\Rightarrow \text{MSSV} \rightarrow \text{TenLop}$

$\rightarrow$ TenLop is transitively dependent on the primary key MSSV, which violates 3NF.

Similarly:

$\text{MaMH} \rightarrow \text{TenMH}$ is also a non-key $\rightarrow$ non-key dependency.

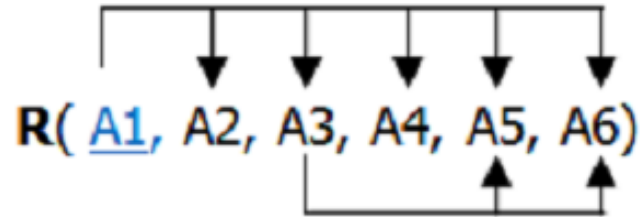
If MaMH is not a full primary key, then this also violates 3NF.

Third Normal Form (3 NF)

Normalization Rules for 3NF:

- Step 1:** Remove transitive dependencies from the relation and separate them into a new relation, with the transitive attribute as the primary key.
- Step 2:** The remaining attributes form a new relation, with the primary key being the same as the original relation's primary key.

Third Normal Form (3 NF)



- Primary key $\{A1\}$; $\{\underline{A1}\} \rightarrow \{A2, A3, A4, A5, A6\}$
- Functional dependency $\{A3\} \rightarrow \{A5, A6\}$
- Relation divide in 2 relation
 - $R1(\underline{A3}, A5, A6)$ - Decompose transitive dependencies
 - $R2(\underline{A1}, A2, A3, A4)$ - The original table remains

Example 1: Converting to 3NF

R (MSSV, TenSV, MaLop, TenLop, MaMH, TenMH, Diem)

Primary key: (MSSV, MaMH)

$F_{R1} = \{\mathbf{MSSV}\} \rightarrow \{\text{TenSV, MaLop}\}$

$F_{R2} = \{\text{MaLop} \rightarrow \text{TenLop}\}$

$F_{R3} = \{\text{MaMH} \rightarrow \text{TenMH}\}$

Divide R in to table:

1. *SINHVIEN(MSSV, TenSV, MaLop)*

Từ FR1: MSSV $\rightarrow$ TenSV, MaLop

2. *LOP(MaLop, TenLop)*

Từ FR2: MaLop $\rightarrow$ TenLop

3. *MONHOC(MaMH, TenMH)*

Từ FR3: MaMH $\rightarrow$ TenMH

4. *KETQUA(MSSV, MaMH, Diem)*

Từ quan hệ chính ban đầu, giữ lại khóa và thuộc tính phụ thuộc trực tiếp vào khóa toàn phần.

Denormalization

A sales system needs to quickly generate customer order reports, including:

- Customer information
- Order information
- Products, quantity, unit price, total amount

After normalization, the tables are:

- CUSTOMER**(CustomerID, Name, Address)
- ORDER**(OrderID, OrderDate, CustomerID)
- ORDER_DETAIL**(OrderID, ProductID, Quantity, UnitPrice)
- PRODUCT**(ProductID, Name, UnitPrice)

Question: What happens when querying this data?

Denormalization

- The normalization process increases the number of tables and relationships.
- When database users encounter performance issues that cannot be resolved by other means (such as database tuning or hardware upgrades), denormalization is required.
- Steps for denormalization:
 - Combine the divided relations to satisfy normalization rules
 - Store redundant data in tables
 - Store summary data in tables

Normalization vs Denormalization

Criteria	Lack of Normalization	Denormalization
Purpose	Poor or incorrect database design	Optimize read performance
Common Issues	Redundant or inconsistent data	Hard to update, repeated information
When to Use	Due to lack of understanding	For read-heavy operations or reporting
Example	Repeated teacher names in classes	Fast sales report generation

Exercise 1

Given the following purchase list table:

InvoiceID	CustomerName	CustomerAddress	ProductID	Product Name	Quantity	UnitPrice
HD01	Nguyễn Văn A	Hà Nội	SP01	Sữa	2	10,000
HD01	Nguyễn Văn A	Hà Nội	SP02	Bánh	1	5,000
HD02	Trần Thị B	Đà Nẵng	SP01	Sữa	1	10,000

Tasks:

1. Analyze and identify **functional dependencies**.
2. **Normalize** the table to **3NF**.
3. Design the **Entity-Relationship (ER) model** after normalization.

Exercise 2

Given the following student and class information table:

MSSV	Tên SV	Lớp	Tên GV CN	Khoa	Mã môn	Tên môn	Điểm
001	An	D21CQ CN01	Cô Mai	CNTT	MH01	CSDL	8.0
002	Bình	D21CQ CN01	Cô Mai	CNTT	MH01	CSDL	6.5
001	An	D21CQ CN01	Cô Mai	CNTT	MH02	LT Java	7.0

Tasks:

1. Identify **functional dependencies**.
2. **Normalize** the data to **BCNF** if possible.
3. Describe the **tables after normalization**.

Exercise 3

Given the following employee and salary information table:

MaNV	Họ tên	Ngày sinh	Phòng	Tên phòng	Hệ số lương	Lương cơ bản	Lương thực lĩnh
NV01	Hoa	1990-01-01	P01	Kế toán	3.0	1,500,000	4,500,000
NV02	Nam	1992-04-12	P02	Kỹ thuật	2.5	1,500,000	3,750,000

Tasks:

1. Identify **redundancies and issues** if the data is not normalized.

2. **Normalize** the table to **3NF** and model the data.

3. Write **SQL queries** after normalization:

- Find employees by department
- Calculate total salary by department

Exercise 4

Given the following library book borrowing management table:

Mã PM	Tên độc giả	Địa chỉ	Tên sách	Tác giả	Ngày mượn	Ngày trả
PM01	Long	Huế	DSA	An	2024-03-01	2024-03-10
PM01	Long	Huế	Java	Bình	2024-03-01	2024-03-10

Tasks:

1. **Analyze** and **normalize** the data to **3NF**.
2. Identify the **benefits of normalization** in this scenario.
3. Design a **preliminary ERD** after normalization.

Exercise 5

Reverse Engineering – From Real-World Requirements:

A company needs to manage **employees, departments, projects, and salaries.**

- Each employee belongs to one department.
- An employee can participate in multiple projects, and each project can have multiple employees.
- Employee salaries change monthly.

Tasks:

1. Create an **initial consolidated data table.**
2. Analyze **functional dependencies.**
3. **Normalize** from 1NF to **3NF** (or **BCNF** if needed).
4. Design the **data model** and **relationships between tables.**

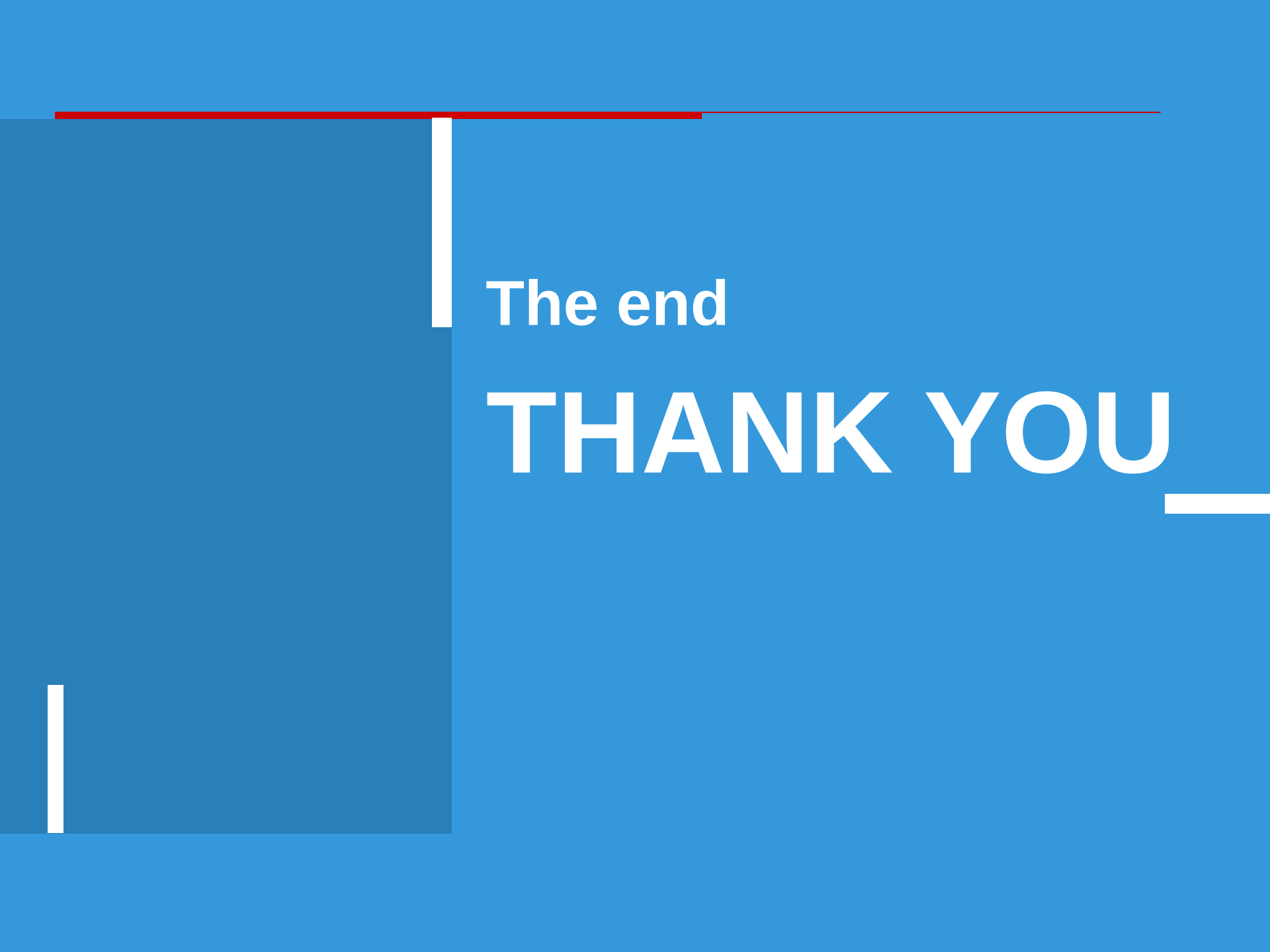
Exercise 6 – Denormalization

Given the following relations already in 3NF:

- **STUDENT**(StudentID, FullName, ClassID)
- **CLASS**(ClassID, ClassName, DeptID)
- **DEPARTMENT**(DeptID, DeptName)
- **RESULT**(StudentID, CourseID, Score)
- **COURSE**(CourseID, CourseName, Credits)

Tasks:

1. **Denormalize** the system into 1 or 2 tables to **reduce JOIN operations** when retrieving student and academic results information.
2. Describe the **advantages and disadvantages** of denormalization in this case.
3. Write an **SQL query** to retrieve: Student name, Class name, Department name, Course name, and Score **after denormalization**.



The end

THANK YOU